ClawPad

Desktop GUI for OpenClaw — zero terminal, zero complexity.

Version	0.1.0 (Phase 1 Beta)
Author	Scott Converse
License	MIT
Date	March 2026
Repository	github.com/scottconverse/clawpad

1. Overview

ClawPad is a native desktop application that wraps the open-source OpenClaw AI agent framework in a visual, point-and-click interface. OpenClaw has amassed over 250,000 stars on GitHub and is one of the most capable open-source agent platforms available, but it demands terminal fluency, manual JSON and Markdown configuration, and comfortable API key management — skills that put it out of reach for most non-technical users. Meanwhile, hosted alternatives like Clawnify, OneClaw, and KiwiClaw charge between $10 and $40 per month for what is fundamentally free software, and they hold your API keys on their servers.

ClawPad fills that gap. It packages the entire OpenClaw setup and management lifecycle into a guided wizard that takes minutes, not hours, from download to a running agent. There is no command line, no config file editing, and no Markdown to write. The user downloads a lightweight installer, runs the wizard, and talks to their agent. Everything stays local: keys are stored in the operating system's native credential store, the agent process runs on the user's machine, and no data leaves the device except what the user's chosen AI provider requires.

The application is built on Tauri v2, producing installers under 700 KB on Windows. It targets Windows, macOS, and Linux, with all three platforms passing CI builds and tests. ClawPad is designed for accessibility from the ground up, meeting WCAG 2.1 AA requirements with full keyboard navigation, screen reader support, and high-contrast theming.

2. Key Features

- **Quick setup wizard** — An 8-screen guided flow that takes a new user from first launch to a running AI agent in minutes, not hours. No terminal, no manual steps.
- **Silent Node.js installation** — Automatically detects existing Node.js installs, nvm, Volta, and Homebrew-managed versions. If Node.js is missing or below v22, ClawPad silently downloads and extracts it to `~/.clawpad/node/` with no elevation prompt on any platform.
- **Visual personality builder** — Choose from role templates (assistant, creative writer, coder, etc.), adjust a tone slider from formal to casual, and watch a live SOUL.md preview update in real time.
- **Secure API key management** — Keys are stored in the OS credential store: DPAPI on Windows, Keychain on macOS, libsecret on Linux. Keys never exist in plaintext at rest.
- **Ollama integration** — Auto-detection of a local Ollama instance, model listing via `/api/tags`, model picker dropdown, no-models guard (prompts user to `ollama pull`), and writes `base_url` (`http://localhost:11434`) and selected model to `openclaw.json`.
- **Channel setup** — Guided configuration flows for Telegram and Discord, with token entry, validation, and connection testing built into the wizard.
- **Skill marketplace with safety ratings** — Browse and install skills from ClawHub with one click. Every skill carries a safety rating: Verified, Community, or Unreviewed.
- **Agent lifecycle management** — The OpenClaw agent runs as a managed child process with automatic restart on crash, periodic health monitoring, and graceful shutdown.
- **Start on login** — Optional system tray launcher that starts the agent at login. Uses a registry key (Windows), launch agent (macOS), or XDG autostart entry (Linux) — no system service required.
- **Resumable setup and uninstall** — Both the setup wizard and the uninstall flow persist their progress to disk as JSON. If the app crashes mid-operation, it picks up where it left off on the next launch.
- **WCAG 2.1 AA accessible** — Full keyboard navigation, ARIA labels, screen reader testing, and high-contrast support throughout the interface.
- **Lightweight** — Windows MSI installer: 672 KB. NSIS setup executable: 415 KB. The entire application leverages the system webview rather than bundling a browser engine.

3. Architecture

System Architecture Diagram

```
+-------------------------------------------------------------+
|                      ClawPad (Tauri v2)                     |
|                                                             |
|  +-------------------------+   +-----------------------+ |
|  |    React Frontend       |   |    Rust Backend       | |
|  |                         |   |                       | |
|  |  Wizard (8 screens)     |   |  installer/           | |
|  |  Dashboard (Phase 2)    |<->|    node.rs            | |
|  |  Components             |IPC|    openclaw.rs         | |
|  |  Lib (types, state)     |   |    skills.rs          | |
|  |                         |   |    uninstall.rs        | |
|  +-------------------------+   |  config/               | |
|                                |    soul.rs             | |
|                                |    openclaw_json.rs    | |
|                                |    env.rs              | |
|                                |  secrets/              | |
|                                |    keychain.rs         | |
|                                |  setup/                | |
|                                |    setup_state.rs      | |
|                                |  process/              | |
|                                |    daemon.rs           | |
|                                |    health.rs           | |
|                                |    autostart/          | |
|                                |  clawhub/              | |
|                                |    registry.rs         | |
|                                |    safety.rs           | |
|                                |  update/               | |
|                                |    openclaw_update.rs  | |
|                                |    skill_update.rs     | |
|                                +-----------------------+ |
+-------------------------------------------------------------+
         |                              |
         v                              v
  +-------------+                +-----------------+
  | System      |                | ~/.clawpad/     |
  | Webview     |                |   node/         |
  | (WRY/Edge/  |                |   secrets/      |
  |  WebKitGTK) |                |   setup-state.json|
  +-------------+                +-----------------+
                                          |
                                          v
                                +-----------------+
```

```
                                  | OpenClaw Agent   |
                                  | (child process)  |
                                  |   ~/.openclaw/   |
                                  |     .env         |
                                  |     SOUL.md      |
                                  |     openclaw.json|
                                  +------------------+
```

Why Tauri v2

ClawPad uses Tauri v2 rather than Electron for three reasons:

1. **Bundle size.** Tauri leverages the operating system's native webview (Edge WebView2 on Windows, WebKitGTK on Linux, WKWebView on macOS) instead of shipping Chromium. This keeps the installer under 700 KB versus the typical 80–150 MB Electron bundle.
2. **Memory footprint.** A Tauri app's baseline memory consumption is roughly 30–50 MB compared to 150–300 MB for an equivalent Electron app, which matters because ClawPad runs alongside the OpenClaw agent process itself.
3. **Security.** Tauri's capability-based permission model restricts the frontend to only the IPC commands explicitly registered in the Rust backend. There is no Node.js runtime in the main process, eliminating an entire class of supply-chain attacks.

Rust Backend Modules

Module	Files	Purpose
`installer/`	`node.rs`, `openclaw.rs`, `skills.rs`, `uninstall.rs`	Downloads, extracts, and verifies Node.js; installs OpenClaw from npm; installs skills from ClawHub; manages uninstall lifecycle
`config/`	`soul.rs`, `openclaw_json.rs`, `env.rs`	Reads and writes SOUL.md personality files, openclaw.json configuration, and the ephemeral .env file
`secrets/`	`keychain.rs`	OS credential store integration — DPAPI on Windows, keyring crate (Keychain/libsecret) on macOS/Linux
`setup/`	`setup_state.rs`	Wizard state persistence, step completion tracking, crash-safe resume
`process/`	`daemon.rs`, `health.rs`, `autostart/`	Child process lifecycle (start/stop/restart), periodic health checks, platform-specific login-item registration
`clawhub/`	`registry.rs`, `safety.rs`	Skill registry client, search, recommendations, and safety rating lookup
`update/`	`openclaw_update.rs`, `skill_update.rs`	Version checking and update logic for OpenClaw and installed skills

Frontend Stack

Technology	Version	Role
React	19	UI framework
TypeScript	5.9	Type safety
Tailwind CSS	v4	Utility-first styling
React Router	v7	Client-side routing
Lucide React	0.577	Icon library
Vite	8	Build tooling and dev server
Vitest	4.1	Unit and component testing

---

4. Security Model

ClawPad treats API keys as the most sensitive data in the system. The security model is designed around the principle that keys should never exist in plaintext at rest.

Secret Materialization Flow

```
1. User enters API key in the wizard UI
         |
         v
2. Frontend sends key to Rust backend via Tauri IPC
         |
         v
3. Rust backend encrypts and stores in OS credential store
   - Windows: DPAPI (ConvertTo-SecureString) --> ~/.clawpad/secrets/
   - macOS:   Keychain (via keyring crate)
   - Linux:   Secret Service / libsecret (via keyring crate)
         |
         v
4. Key stored at rest (encrypted by OS, never plaintext)
         |
   (when daemon starts)
         |
         v
5. Rust backend retrieves key from credential store
         |
         v
6. Writes ephemeral .env file to ~/.openclaw/.env
   - File permissions set to 600 (owner read/write only)
   - Windows: icacls restricts to current user
         |
         v
7. OpenClaw agent process reads .env on startup
         |
   (when daemon stops)
         |
         v
8. .env file is scrubbed (deleted) immediately
```

Additional Security Measures

- **Crash recovery scrub.** On every ClawPad launch, the Rust backend checks for and removes any stale `.env` file left behind by a prior crash.
- **SHA256 checksum verification.** The Node.js download is verified against the official `SHASUMS256.txt` from nodejs.org before extraction.
- **Windows file permissions.** The `.env` file and secret storage directory are locked to the current user via `icacls` with inheritance removed.

- **Tauri capability restrictions.** The frontend can only invoke IPC commands that are explicitly registered in the `invoke_handler`. There is no shell access, no arbitrary filesystem access, and no network access from the webview context.
- **No telemetry.** ClawPad collects and transmits no usage data by default.

---

5. Project Structure

```
clawpad/
├── .github/
│   └── workflows/
│       ├── build.yml              # Cross-platform build (Ubuntu, macOS, Windows)
│       ├── test.yml               # Cross-platform test suite
│       ├── release.yml            # Tag-triggered release with GitHub assets
│       └── compat.yml             # Nightly OpenClaw compatibility check
├── e2e/
│   ├── helpers/                   # Shared test utilities
│   ├── wizard-walkthrough.e2e.ts  # 8 wizard walkthrough E2E tests (WebdriverIO)
│   ├── accessibility.e2e.ts       # 7 accessibility E2E tests (WebdriverIO)
│   ├── autostart.spec.ts          # Autostart toggle E2E tests (old stubs, superseded)
│   ├── channel-setup.spec.ts      # Channel configuration E2E tests (old stubs, superse
│   ├── resume-wizard.spec.ts      # Crash recovery resume E2E tests (old stubs, superse
│   ├── secret-lifecycle.spec.ts   # API key lifecycle E2E tests (old stubs, superseded)
│   ├── setup-wizard.spec.ts       # Full wizard walkthrough E2E tests (old stubs, super
│   └── uninstall.spec.ts          # Uninstall flow E2E tests (old stubs, superseded)
├── src/                           # React frontend
│   ├── __tests__/
│   │   ├── components/
│   │   │   ├── SafetyBadge.test.tsx
│   │   │   └── ToneSlider.test.tsx
│   │   ├── lib/
│   │   │   └── wizard-state.test.ts
│   │   ├── wizard/
│   │   │   ├── ModelProvider.test.tsx
│   │   │   └── Personality.test.tsx
│   │   └── setup.ts               # Test setup (mocks)
│   ├── components/
│   │   ├── ChannelCard.tsx         # Channel selection card
│   │   ├── ProgressBar.tsx         # Animated progress indicator
│   │   ├── ProviderCard.tsx        # AI provider selection card
│   │   ├── SafetyBadge.tsx         # Skill safety rating badge
│   │   ├── SkillCard.tsx           # Skill browser card
│   │   ├── StatusIndicator.tsx     # Agent status light
│   │   └── ToneSlider.tsx          # Personality tone range slider
│   ├── dashboard/
│   │   ├── DashboardLayout.tsx     # Dashboard shell (Phase 2)
│   │   └── Overview.tsx            # Agent overview panel (Phase 2)
│   ├── lib/
│   │   ├── tauri.ts               # Tauri IPC wrappers
│   │   ├── templates.ts           # Personality role templates
│   │   ├── types.ts               # Shared TypeScript types
│   │   └── wizard-state.ts        # Wizard state management
│   ├── wizard/
```

```
│   │   │   ├── Welcome.tsx           # Screen 1: Welcome + system check
│   │   │   ├── Dependencies.tsx       # Screen 2: Node.js detection/install
│   │   │   ├── Install.tsx            # Screen 3: OpenClaw installation
│   │   │   ├── ModelProvider.tsx      # Screen 4: AI provider + API key
│   │   │   ├── Personality.tsx        # Screen 5: Role, tone, SOUL.md
│   │   │   ├── Channels.tsx           # Screen 6: Telegram/Discord setup
│   │   │   ├── Skills.tsx             # Screen 7: Skill browser + install
│   │   │   ├── Done.tsx               # Screen 8: Summary + launch
│   │   │   └── WizardLayout.tsx       # Wizard chrome (progress bar, nav)
│   │   ├── styles/                    # Global CSS
│   │   ├── App.tsx                    # Root component + router
│   │   ├── main.tsx                   # React entry point
│   │   └── vite-env.d.ts             # Vite type declarations
├── src-tauri/
│   ├── src/
│   │   ├── bin/
│   │   │   ├── spike_daemon.rs    # Daemon spike binary
│   │   │   ├── spike_dpapi.rs     # DPAPI spike binary
│   │   │   ├── spike_install.rs   # Node install spike binary
│   │   │   ├── spike_kc2.rs       # Keychain v2 spike binary
│   │   │   ├── spike_keychain.rs  # Keychain spike binary
│   │   │   └── spike_node.rs      # Node detection spike binary
│   │   ├── installer/
│   │   │   ├── mod.rs
│   │   │   ├── node.rs            # Node.js detection + silent install
│   │   │   ├── openclaw.rs        # OpenClaw npm installation
│   │   │   ├── skills.rs          # Skill installation from ClawHub
│   │   │   └── uninstall.rs       # Resumable uninstall engine
│   │   ├── config/
│   │   │   ├── mod.rs
│   │   │   ├── soul.rs            # SOUL.md read/write/generate
│   │   │   ├── openclaw_json.rs   # openclaw.json configuration
│   │   │   └── env.rs             # Ephemeral .env management
│   │   ├── secrets/
│   │   │   ├── mod.rs
│   │   │   └── keychain.rs        # DPAPI / Keychain / libsecret
│   │   ├── setup/
│   │   │   ├── mod.rs
│   │   │   └── setup_state.rs     # Wizard state persistence
│   │   ├── process/
│   │   │   ├── mod.rs
│   │   │   ├── daemon.rs          # Child process lifecycle
│   │   │   ├── health.rs          # Periodic health checks
│   │   │   └── autostart/
│   │   │       ├── mod.rs
│   │   │       ├── windows.rs     # Registry Run key
│   │   │       ├── macos.rs       # LaunchAgent plist
│   │   │       └── linux.rs       # XDG autostart / systemd
│   │   ├── clawhub/
```

```
│   │   │   │   │   ├── mod.rs
│   │   │   │   │   ├── registry.rs        # Skill search + recommendations
│   │   │   │   │   └── safety.rs          # Safety rating lookup
│   │   │   │   ├── update/
│   │   │   │   │   ├── mod.rs
│   │   │   │   │   ├── openclaw_update.rs # OpenClaw version check + update
│   │   │   │   │   └── skill_update.rs    # Skill version check + update
│   │   │   │   ├── lib.rs                 # Module registration + Tauri setup
│   │   │   │   └── main.rs                # Binary entry point
│   │   │   ├── tests/
│   │   │   │   ├── fixtures/              # Test fixture data
│   │   │   │   ├── installer_test.rs      # Node installer integration tests
│   │   │   │   ├── setup_state_test.rs    # Setup state persistence tests
│   │   │   │   ├── daemon_test.rs         # Daemon lifecycle tests
│   │   │   │   ├── keychain_test.rs       # OS keychain tests (ignored by default)
│   │   │   │   ├── autostart_test.rs      # Platform-specific autostart tests
│   │   │   │   └── uninstall_test.rs      # Uninstall state persistence tests
│   │   │   ├── Cargo.toml                 # Rust dependencies
│   │   │   └── Cargo.lock
│   ├── wdio.conf.ts                       # WebdriverIO E2E test configuration
│   ├── package.json                       # Node dependencies + scripts
│   ├── vite.config.ts                     # Vite configuration
│   ├── tsconfig.json                      # TypeScript configuration
│   ├── SPIKE-RESULTS.md                   # Node.js installer spike findings
└── README.md                              # Project README (concise version)
```

---

6. Development Guide

Prerequisites

Requirement	Details
Node.js	>= 22 (nodejs.org)
Rust	Stable toolchain (rustup.rs)
Windows	Visual Studio Build Tools with C++ workload
macOS	Xcode Command Line Tools
Linux (Ubuntu)	`libwebkit2gtk-4.1-dev`, `libappindicator3-dev`, `librsvg2-dev`

Initial Setup

```
git clone https://github.com/scottconverse/clawpad.git
cd clawpad
npm install
```

Development Server

Start the Tauri dev server with hot-reload for both the frontend and backend:

```
npx tauri dev
```

This compiles the Rust backend, starts the Vite dev server for the React frontend, and opens the application window. Changes to `.tsx` files trigger instant HMR; changes to `.rs` files trigger a Rust recompile.

Code Structure Conventions

- **Frontend IPC calls** go through `src/lib/tauri.ts`, which wraps `@tauri-apps/api/core.invoke()` with typed function signatures.
- **Rust commands** are `#[tauri::command]` async functions registered in `lib.rs`.
- **Shared types** are defined in `src/lib/types.ts` (frontend) and as `serde`-derived structs in each Rust module.

Running Tests

```
# Frontend unit and component tests (vitest)
npx vitest run

# Rust integration tests (skips ignored tests)
cd src-tauri && cargo test

# Rust ignored tests (requires real system state: keychain, network, etc.)
cd src-tauri && cargo test -- --ignored

# E2E tests (requires built app + msedgedriver for Edge WebView2)
npm run test:e2e

# Watch mode for frontend tests
npx vitest
```

Building for Production

```
npx tauri build
```

This compiles an optimized Rust binary, bundles the Vite-built frontend, and produces platform-specific installers in `src-tauri/target/release/bundle/`.

Useful Commands

Command	Purpose
`npx tauri dev`	Dev server with hot-reload
`npx vitest run`	Run frontend tests once
`npx vitest`	Run frontend tests in watch mode
`cd src-tauri && cargo test`	Run Rust tests
`cd src-tauri && cargo test -- --ignored`	Run ignored Rust tests
`npm run test:e2e`	Run E2E tests (WebdriverIO + tauri-driver)
`cd src-tauri && cargo check`	Fast Rust type-check without building
`npx tauri build`	Production build with installers
`npm run build`	Build frontend only (Vite)

---

7. Testing

Test Summary

Layer	Framework	Count	Status
Frontend unit/component	Vitest + Testing Library	19	Passing
Rust integration	Cargo test	12	Passing
Rust (ignored, require real state)	Cargo test --ignored	7	Passing
End-to-end	WebdriverIO v9 + tauri-driver + msedgedriver	15	Passing
Total		**53**	**0 failures**

Frontend Tests (Vitest)

Tests are located in `src/__tests__/` and cover:

- **Component tests** — `ToneSlider` (rendering, interaction, ARIA attributes), `SafetyBadge` (rating display)
- **Wizard screen tests** — `ModelProvider` (provider card rendering, validation gating), `Personality` (role cards, tone slider, preview updates)
- **Library tests** — `wizard-state` (initial state, step detection, completion logic)

Run with: `npx vitest run`

Rust Integration Tests (Cargo)

Tests are located in `src-tauri/tests/` and cover:

- **installer_test** — Node.js detection, version manager detection, ClawPad node path validation
- **setup_state_test** — Fresh state generation, step completion persistence
- **daemon_test** — Stale `.env` scrubbing, daemon status when not running
- **uninstall_test** — Uninstall state serialization, partial uninstall resume detection
- **autostart_test** — Default-disabled check (platform-gated)

Run with: `cd src-tauri && cargo test`

Ignored Tests (Require Real System State)

These tests are marked `#[ignore]` because they interact with real OS resources:

- **keychain_test** (3 tests) — Store/retrieve round-trip, delete, list. Interacts with the real OS keychain. Uses a Mutex for serial execution to avoid race conditions on the shared DPAPI key index.
- **installer_test** (2 tests) — Full Node.js download and install. Requires network and writes to `~/.clawpad/node/`.
- **autostart_test** (1 test per platform) — Enable/disable round-trip. Modifies registry keys, launch agents, or systemd units.
- **setup_state_test** (1 test) — `verify_all_steps` calls real system checks.

Run with: `cd src-tauri && cargo test -- --ignored`

End-to-End Tests (WebdriverIO)

15 real E2E tests launch the compiled application using WebdriverIO v9 with tauri-driver and msedgedriver (Edge WebView2). Configuration is in `wdio.conf.ts`. Run with `npm run test:e2e`.

- `e2e/wizard-walkthrough.e2e.ts` (8 tests) — Full wizard walkthrough from Welcome screen through Done, exercising each step of the setup flow.
- `e2e/accessibility.e2e.ts` (7 tests) — WCAG 2.1 AA compliance checks including keyboard navigation, ARIA labels, focus management, and screen reader compatibility.

The older stub files (`setup-wizard.spec.ts`, `secret-lifecycle.spec.ts`, `channel-setup.spec.ts`, `resume-wizard.spec.ts`, `autostart.spec.ts`, `uninstall.spec.ts`) still exist in `e2e/` but are superseded by the real test suites above.

---

8. Build Artifacts

What `npx tauri build` Produces

Platform	Artifact	Format	Notes
Windows	MSI installer	`.msi`	Standard Windows Installer package
Windows	NSIS installer	`.exe`	Lighter-weight NSIS setup executable
macOS	App bundle	`.app` in `.dmg`	Universal binary (Intel + Apple Silicon)
Linux	AppImage	`.AppImage`	Portable, no install required
Linux	Debian package	`.deb`	For apt-based distributions

Verified Windows Build Sizes

Artifact	Size
`ClawPad_0.1.0_x64_en-US.msi`	672 KB
`ClawPad_0.1.0_x64-setup.exe`	415 KB

These sizes reflect the Tauri advantage: no bundled browser engine. The application uses the system's Edge WebView2 (Windows), WebKitGTK (Linux), or WKWebView (macOS).

Output directory: `src-tauri/target/release/bundle/`

---

9. CI/CD

Four GitHub Actions workflows automate building, testing, releasing, and compatibility monitoring.

Workflows

Workflow	File	Trigger	Platforms	Purpose
Build	`build.yml`	Push/PR to `main`	Ubuntu, macOS, Windows	Compiles the full Tauri application on all three platforms. Catches compilation errors before merge.
Test	`test.yml`	Push/PR to `main`	Ubuntu, macOS, Windows	Runs Rust integration tests (`cargo test`), frontend unit tests (`vitest`), and E2E tests (`npm run test:e2e`) on all platforms. All passing.
Release	`release.yml`	Tag push (`v*`)	Ubuntu, macOS, Windows	Builds platform-specific installers via `tauri-action` and publishes them as GitHub Release assets.
Compatibility	`compat.yml`	Nightly (06:00 UTC) + manual	Ubuntu	Installs the latest OpenClaw from npm and runs the Rust test suite against it. On failure, automatically creates a GitHub Issue tagged `bug` + `compatibility` .

Dependency Caching

All workflows cache:

- `~/.cargo/registry` and `~/.cargo/git` (Cargo registry)
- `src-tauri/target` (Rust build artifacts)
- `node_modules` (via `setup-node` cache)

Cache keys are based on `Cargo.lock` hash for Rust and npm lockfile for Node.

---

10. Roadmap

Phase 1: Setup Wizard (Current — Shipped)

- 8-screen setup wizard from download to running agent
- Silent Node.js detection and installation
- OpenClaw installation from npm
- API key management with OS credential store
- Personality builder with SOUL.md preview
- Channel configuration (Telegram, Discord)
- Skill browser with safety ratings
- Agent lifecycle management (start, stop, restart, health monitoring)
- System tray autostart
- Resumable setup and uninstall
- Accessibility (WCAG 2.1 AA)
- Cross-platform CI/CD with nightly compatibility checks

Phase 2: Dashboard and Ecosystem

- **Agent dashboard** — Real-time status, log viewer, conversation history
- **WhatsApp integration** — Deferred from Phase 1 due to Baileys library fragility; evaluating alternatives
- **Skill marketplace** — Full ClawHub integration with install, update, and review flows
- **Multi-agent support** — Run and manage multiple OpenClaw instances

- **Settings panel** — Advanced configuration without editing JSON

Phase 3: Future Considerations

- **Auto-update** — Silent background updates for ClawPad itself via Tauri's updater
- **Plugin system** — Community-developed UI extensions
- **Team management** — Shared configurations for organizations
- **Analytics dashboard** — Usage metrics and conversation insights (local only)
- **Mobile companion** — Status monitoring and quick replies from mobile devices

---

11. Competitive Positioning

```
                More Control / Local

                    * ClawPad
                 (free, local, GUI)

   Easier ———                          ——— Harder

   Clawnify      OneClaw      KiwiClaw
   ($$$)         ($10/mo)     ($15-39/mo)

                  OpenClaw CLI
                (free but terminal)

                Less Control / Cloud
```

Product	Price	Local/Cloud	GUI	Terminal Required
ClawPad	Free	Local	Yes	No
OpenClaw CLI	Free	Local	No	Yes
Clawnify	$20+/mo	Cloud	Yes	No
OneClaw	$10/mo	Cloud	Yes	No
KiwiClaw	$15–39/mo	Cloud	Yes	No

ClawPad occupies the unique position of being both free and easy to use while keeping all data local. The only product in the “free + local” space is the OpenClaw CLI, which requires terminal proficiency. All GUI alternatives are cloud-hosted and paid.

---

12. Known Limitations

- **OpenClaw is fictional.** This project is a proof-of-concept demonstrating the architecture, UI patterns, and security model for wrapping a complex CLI tool in a native desktop GUI. The “OpenClaw” agent framework, “ClawHub” skill registry, and competing services referenced throughout are not real products.
- **Some Rust modules contain `todo!()` stubs.** Modules that depend on external APIs (ClawHub registry, OpenClaw update endpoints) have stubbed implementations that return mock data. The module interfaces and IPC contracts are final; only the HTTP calls need to be connected.
- **WhatsApp integration deferred.** The original PRD included WhatsApp as a channel option. This was deferred because the Baileys library (the primary open-source WhatsApp Web API) is fragile and breaks frequently with WhatsApp protocol changes.
- **No code signing.** Windows and macOS builds are not code-signed, which will trigger OS security warnings on first launch. Code signing certificates are needed for production distribution.

---

13. License

MIT License

Copyright © 2026 Scott Converse

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.